Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Sıralama probleminin biçimsel belirtimi (ekrandan, 00:10:00)
  - Girdi X bir tam sayı vektörü, çıktı Y ise X'in artan sırada sıralanmış hâlidir. Basit bir problemde bile kesin bir tanım yapmak için pek çok uç durum düşünülmelidir.
  - "Y'deki her eleman bir sonrakinden küçüktür" tanımı eşit elemanları dışarıda bırakır; "küçük veya eşittir" olmalıdır. Son elemanın sonrası olmadığı için bu kural son elemandan bir öncekine kadar uygulanır.
  - Girdiye olan duyarlılık: yalnız çıktının sıralı olmasını istemek yetmez. Slayttaki karşı örnekler: 3, 2, 1 girdisine 4, 5, 6 çıktısı; 1, 1, 1, 1, 2, 3 çıktısı; 1, 2, 3, 4 çıktısı. Hepsi sıralıdır ama girdinin sıralanmış hâli değildir. Çıktının içeriği girdiyle birebir aynı olmalı, her eleman aynı sayıda yer almalı ve yeni eleman bulunmamalıdır.
  - Döngüsellik (circularity): bir kavram kendi terimleriyle tanımlanmamalıdır. "Y sıralı olmalıdır" ifadesi sıralamayı sıralamayla tanımlar; bunun yerine "her eleman sonrakinden küçük veya eşittir" denmelidir. Hocanın benzetmesi: "bardak, bardak tipindeki şeylere denir" gibi bir tanım hiçbir şey katmaz.
- Biçimsel belirtimin üç parçası
  - İmza (signature): adı, girdinin ve çıktının adları ve tipleri. Sıralama için girdi `Vector<int> x`, çıktı `y = sort(x)`, yine bir tam sayı vektörü. Ön ve son koşullarda bu adlar kullanılır.
  - Ön koşullar: girdiler hakkındaki savlardır; işlemin anlamlı çalışması için neyin doğru olması gerektiğini söyler. Sıralamada ön koşul gerekmez (true); boş vektörün sıralanmış hâli de boş vektördür. Karekökte $x \geq 0$ ön koşulu gerekir (kompleks sayılarla çalışılsaydı gerekmeyebilirdi).
  - Son koşullar: işlem bittikten sonra neyin garanti edildiğini, çıktının girdiyle nasıl ilişkili olduğunu söyler (ekrandan, 00:20:00). Karekökte $Y = SQRT(X)$ demek döngüsel tanımdır; doğrusu $Y \cdot Y = X$.
  - Her işlem saf fonksiyon değildir: global değişkene, başka nesneye ya da veritabanına yazmak gibi yan etkiler de son koşulla tanımlanmalıdır.
- Sıralama için son koşullar (ekrandan, 00:24:30)
  - Sıralılık: $\forall i : 1 \leq i \leq n-1,\ y[i] \leq y[i+1]$. $n = 0$ ise hiç i oluşmaz; ifade boş yere doğru olur ama son koşulun doğruluğunu değiştirmez. Üst sınır $n-1$'dir, çünkü n alınırsa $i+1$ vektörün dışına taşar.
  - `Sıralı(x): Boolean` gibi bir imzanın son koşulu yalnız bu ifadedir; ters sıralı için $\geq$ kullanılır.
  - İçerik koşulu permütasyonla verilir: Y, X'in bir permütasyonuysa her eleman X'ten gelir, X'teki her eleman Y'dedir ve tekrar sayıları aynıdır.
- Permütasyonun özyinelemeli tanımı (ekrandan, 00:30:00 ve 00:33:00)
  - 1. durum: iki vektör de boşsa ($|X| = 0$) permütasyondur.
  - 2.1: ilk elemanlar aynıysa, kalanların permütasyon olup olmadığına bakılır.
  - 2.2: ilk elemanlar farklıysa, $X[1]$ Y'de bir j konumunda aranır; X'in kalanı ile Y'nin j dışındaki parçalarının birleşimi permütasyon olmalıdır.
  - Sıralamanın tam son koşulu: sıralılık ifadesi ve `Permütasyon(x, y)`.
- OCL'ye ilk bakış: `Sıralı()` belirtimi (ekrandan, 00:35:30)
  - `context Vector::Sıralı() : Boolean`, `pre: true`, `post: self->forAll(i:Integer | i > 0 and i < self.length implies self.at[i] <= self.at[i+1])`.
  - `|` (pipe) değişkeni önermeden ayırır; `implies`, FOL'deki çıkarımın karşılığıdır. OCL'de

vektör yerine sequence vardır.

- OCL'nin genel özellikleri (ekrandan, 00:40:00)
  - UML'in resmî bir parçasıdır. Diyagramlar her şeyi anlatamaz; tasarımda daha ayrıntı gerektiren noktalar OCL ile kesinleştirilir.
  - Belirtim dilidir, bildirimseldir (declarative): yapılacak adımları değil, sonuçları tanımlar. Atama yoktur; = eşitlik ya da bildirimdir.
  - Güçlü tiplidir (strongly typed): değişkenler ve tipleri bellidir, bir tip başkasının yerine kullanılamaz.
  - Destekleyen araçlar: Rational Rose, ArgoUML, Eclipse (eklentileri), Poseidon/Octopus, Enterprise Architect.
  - FOL'e doğrudan dönüştürülebilir ama programlama diline yakın sözdizimi sayesinde daha kolay okunur.
- OCL ile yazılabilenler
  - Sınıf değişmezleri, işlemlerin ön ve son koşulları, türetilen özelliklerin nasıl hesaplanacağı (ör. yaş, faiz), durum diyagramı geçişleri için guard'lar, mesaj hedefleri, stereotipler için tip değişmezleri ve sınıf diyagramından türeyen nesneler üzerinde sorgular.
- OCL kısıtının sözdizimi
  - `context <tanımlayıcı> <kısıt tipi>: <Boolean ifade>`. Context bir sınıf (`context Ziyaretçi`) ya da bir işlem (`Ziyaretçi::ödünçAldıkları`) olabilir.
  - Kısıt tipleri: `inv` (değişmez), `pre` (ön koşul), `post` (son koşul).
- Değişmezler (invariant) (ekrandan, 00:50:00)
  - Bir özelliğin her zaman doğru olan ifadesidir; sistemin anahtar bir gereksinimi gibi düşünülebilir. Örnek: `context BüyükŞirket inv: çalışanSayısı > 50`.
  - Veritabanındaki bütünlük kısıtlarına (integrity constraints) benzer: şirket silinince içindeki işçinin ne olacağı, avare işaretçi (dangling pointer) durumu.
- Ön ve son koşullar
  - Karekök: context tam sayı sınıfının karekök işlemi, `pre: self >= 0`, `post: self = result * result`. `self` context nesnesi, `result` dönüş değeridir.
  - Özellik değeri değiştiren işlem (ekrandan, 00:59:05): `context Account::deposit(Real : amount)`, `pre: amount > 0`. `post: balance = balance + amount` yanlıştır, çünkü = eşitliktir. Doğrusu `post: balance = balance@pre + amount`; `@pre`, işlem çalışmadan önceki değerdir.
  - Takas (swap) örneği (ekrandan, 01:00:00): `post: a = b@pre and b = a@pre`; geçici değişkene gerek yoktur.
- OCL basit veri tipleri (ekrandan, 01:10:00)
  - Boolean (true, false; and, or, xor, not, implies, if-then-else), Integer (*, +, -, /, `abs()`), Real (*, +, -, /, `floor()`), String (`toUpper()`, `concat()`).
- OCL anahtar kelimeleri (ekrandan, 01:20:00)
  - `inv`, `pre`, `post`; `if-then-else-endif`; `not`, `or`, `and`, `xor`, `implies`; `package`, `endpackage`; `context`; `def` (global tanım); `let`, `in` (yerel tanım); `derive` (özellik türetme); `init` (ilk değer); `result`; `self`.
  - `let` örneği (ekrandan, 01:24:20): `let income : Integer = self.job.salary->sum() in if isUnemployed then income < 100 else income >= 100 endif`. Uzun bir ifadeyi kısaltıp tekrar tekrar yazmamayı sağlar.
- Gezinti (navigation) (ekrandan, 01:30:00)
  - Context sınıfı dışındaki bir sınıfın özelliğine, diyagramda erişilebilir olmak şartıyla, noktayla ilerleyerek ulaşılır: `context Customer` içinde `self.order.date`.
  - Çoğullama varsa (1-n, m-n) gezintinin sonucu tek nesne değil bir koleksiyondur. Koleksiyon işlemi `->` ile ayrılır (ekrandan, 01:31:20): `self.Order.Check.bankID->size()`.
- Koleksiyonlar ve diğer özellikler
  - Dört koleksiyon tipi: Collection, Set (tekrarsız), Bag, Sequence (sıralı). İşlemler: `size`, `includes`, `count`, `includesAll`, boş/dolu kontrolü, `sum`, `exists`, `forAll`, `iterate`. Alt tipler bu işlemleri miras alır.
  - Daha az kullanılanlar: tuple (kayıt benzeri değişken grubu), enumeration, mesaj ifadeleri (sinyal gönderildi belirtimi, işlem tetikleme), UML meta modeline erişim ve otomatik düzleştirme (automatic flattening: iç içe koleksiyonlarda iki seviyeyi ayrı ayrı belirtmeye gerek kalmaz).
- Kütüphane örneğine OCL yazmak (ekrandan, 01:40:00)

- Diyagramdaki tanımı destekleyen ya da diyagramda gösterilemeyen yeni bir tanım getiren ifadeler OCL ile yazılır. Bilginin varlığını söyleyen gereksinimler (ad, adres, telefon) özelliklerle; ödünç alınamayan parçalar genelleştirmeyle zaten karşılanmıştı.
- Gereksinim 6 (ekrandan, 01:43:40): `context Kitap inv: if cokSatan then oduncPeriyodu = 2 --hafta else oduncPeriyodu = 3 endif`. DVD için: `context DVD inv: oduncPeriyodu = 2`.
- Gereksinim 3 (ekrandan, 01:49:30): `context Ziyaretci::oduncAldiklari() : Set(OduncVerilebilirParca), post: result = Odunc.oduncVerilebilirParca`. Sorgu işlemi olduğu için ön koşul yoktur (true). Küme kullanılır, çünkü aynı kopya aynı anda iki kez ödünç alınamaz.
- Gereksinim 4 (ekrandan, 01:52:20): `context Ziyaretci::oduncAl(i: oduncVerilebilirParca), pre: yas <= 12 implies oduncAldiklari()->size() < 5`. Kontrol her ödünç alma işleminde yapılmalıdır, bu yüzden ön koşuldur. `<=` değil `<` kullanılır.
- Diğer ön koşullar: parçanın mevcut olması ve başka birinin istememiş olması (talep listesinde ilk sırada olmak). Bunlar için yeni işlemler gerekir.
- `oduncAl` son koşulu 1 (ekrandan, 01:59:00): `post: exists(o: Odunc | o.oduncVerilebilirParca = i and o.teslimTarihi = os.getDate() + o.oduncVerilebilirParca.oduncPeriyodu and Odunc = Odunc@pre -> including(o))`.
- `oduncAl` son koşulu 2 (ekrandan, 02:01:50): `post: let b : Baslik = i.baslik in if b.talepler.ziyaretci -> includes(self) then talepler = talepler@pre->reject(baslik=b and ziyaretci=self) else true endif`. `implies` ile de yazılabilirdi.
- Türetilen özellik (ekrandan, 02:03:40): `context Ziyaretci::yas: Date`, `derive: OperatingSystem.getDate() - doğumTarihi`.
- Eksik kalanlar: gereksinim 1, 3, 8, 10 ve 11; işlevsel olmayan gereksinimler; mevcut mu ve başkası istemedi mi işlemleri; iade ve ceza ödeme; parça ve ziyaretçilerin yaratılması.

Hocanın Özellikle Vurguladığı Kısımlar

- Basit bir problemde bile kesin tanım çok düşünmeyi gerektirir
  - Sıralamada yalnız çıktının sıralı olduğunu söylemek yetmez; girdiyle çıktı arasındaki ilişki (aynı elemanlar, aynı tekrar sayısı, yeni eleman yok) da tanımlanmalıdır. Askerî ya da güvenlik yazılımlarında bu kesinlik zorunludur.
  - Başta harcanan bu emek, anlaşılmayan gereksinimler yüzünden tekrar tekrar çalışmaktan kurtarır.
- Döngüsel tanımdan kaçınmak
  - Son koşulda fonksiyonun kendisi kullanılmaz ($Y = SQRT(X)$ değil $Y \cdot Y = X$); girdi ile çıktı arasındaki ilişki tanımlanır.
- OCL'de = atama değildir
  - `balance = balance + amount` yanlış bir eşitliktir. Değer değişiyorsa `@pre` ile işlemden önceki değer kullanılır. Swap'te de geçici değişken yoktur.
- Sayı görüyorsanız değişmez yazın
  - Gereksinimde sabit sayılar (2 hafta, 3 hafta, 10 TL) varsa bunlar genellikle değişmez olarak tanımlanır. Aynı kısıt son koşul olarak da yazılabilir; tek bir doğru yol yoktur ama en basiti tercih edilmelidir.
- `< 5`, `<= 5` değil
  - Çocuk sınırında ön koşul her ödünç almadan önce kontrol edilir; `<=` olursa altıncı parça alınabilir.
- Belirtim "nasıl"a karışmaz
  - OCL ile metodun ne yapacağı ve neyi garanti edeceği kısıtlanır; uygulayıcı nasıl yaparsa yapsın bu koşullar doğru olmalıdır. Tasarımı alan herkes aynı şeyi anlar.
- OCL sıkı düşünmeyi sağlar
  - Analiz ilerledikçe yeni gereksinimler ve açık noktalar ortaya çıkar; bunlar gerçekleştirmeye geçmeden, sürecin erken aşamasında fark edilir. Gerekirse müşteriyle yeniden görüşülür.

Kısa Tekrar Notları

- Biçimsel belirtim = imza + ön koşullar + son koşullar.
- Ön koşul girdiler hakkında, son koşul çıktı ve ortam hakkında garantidir. Sorgu işlemlerinde ön koşul genelde true'dur.
- Sıralama son koşulu: $\forall i : 1 \leq i \leq n-1,\ y[i] \leq y[i+1]$ ve `Permütasyon(x, y)`.
- Permütasyon özyinelemeli tanımlanır: boşsa doğru; ilk elemanlar eşitse kalanlar; değilse $X[1]$'in Y'deki yeri bulunup geri kalanlar.
- OCL: UML'in parçası, bildirimsel, güçlü tipli, yalnız kısıt tanımlar.
- Sözdizimi: `context <sınıf ya da Sınıf::işlem> inv|pre|post: <Boolean ifade>`.
- `self` context nesnesi, `result` dönüş değeri, `x@pre` işlemden önceki değer.
- Anahtar kelimeler: `def`, `let ... in`, `derive`, `init`, `if-then-else-endif`, `implies`.
- Gezinti `.` ile, koleksiyon işlemi `->` ile yapılır; çoğullama varsa sonuç koleksiyondur.
- Koleksiyonlar: Collection, Set, Bag, Sequence; `size`, `includes`, `forAll`, `exists`, `including`, `reject`, `sum`.
- Kütüphane: Kitap için `if cokSatan then oduncPeriyodu = 2 else oduncPeriyodu = 3 endif`; DVD için `oduncPeriyodu = 2`; çocuk için `pre: yas <= 12 implies oduncAldiklari()->size() < 5`.

Detaylı Açıklamalar

- Sıralamanın belirtimi neden zordur?
  - Hoca konuya iskambil kâğıtlarını elde sıralama örneğiyle girer: insan bunu sezgisel yapar ama bilgisayara nasıl yapılacağını söylemek gerekir. İlk Türkçe tanım "X tam sayı vektörü verildiğinde Y tam sayı vektörü döndürülür ve Y'deki her eleman bir sonrakinden küçüktür" olur. Eşit elemanlar yüzünden "küçük veya eşit" denmelidir; son elemanın sonrası olmadığından kural son elemandan bir öncekine kadar uygulanır.
  - Girdinin tipini (tam sayı vektörü) belirtmek girdi kısıtını kesinleştirir; aksi hâlde elmalar, armutlar da sıralanması beklenen girdi olurdu.
  - Asıl tuzak girdiye olan duyarlılıktır. 3, 2, 1 girdisine 4, 5, 6 üreten bir program sıralı çıktı koşulunu sağlar. "Çıktıdaki her eleman girdide yer almalıdır" eklenince 1, 1, 1, 1, 2, 3 çıktısı kabul edilir. Tekrar sayısı koşulu eklenince bu kez 1, 2, 3, 4 gibi fazladan eleman içeren bir çıktı tartışılır; bu yüzden "başka eleman yer almamalıdır" da eklenir. Sonuçta bu üç koşulun hepsini tek seferde sağlayan matematiksel kavram permütasyondur.
- Ön ve son koşul seçimi
  - Sıralamada ön koşul yoktur; isterseniz `true` yazılır. Boş vektör de sıralanabilir, bu yüzden $|x| > 0$ koşulu koymak yerine boş kümeyi kapsamak daha doğrudur.
  - Karekökte ön koşul $x \geq 0$'dır. Hoca, negatif girdide istisna üreten bir sürüm de tanımlanabileceğini ama ön koşulu garanti etmenin daha doğru bir yaklaşım olduğunu söyler.
  - Son koşul, çalışma bittikten sonra ortamın ne hâle geldiğini söyler. Saf fonksiyonlarda bu yalnız çıktıyla girdi arasındaki ilişkidir; yan etkisi olan işlemlerde değişen özellikler de son koşulda yer alır.
- Permütasyonun son koşulu (ekrandan, 00:33:00)

```
Permütasyon(X,Y) <=>
  |X| = 0  V
  ( |X| > 0 =>
      ( X[1] = Y[1] ∧ Permütasyon(kalan(X), kalan(Y)) )  V
      ( X[1] ≠ Y[1] ∧
        ( ∃j: 1 < j ≤ |Y| • ( X[1] = Y[j] ) ∧
          Permütasyon( kalan(X), (Y[1..j-1] ++ Y[j+1..|Y|]) ) ) ) )
```

  - Kalan her adımda küçülür ve sıfır uzunluk durumu ele alındığı için özyineleme sonlanır. Örneğin $X[1] = 5$ ise 5'in Y'de bir yerde olması gerekir; Y bu noktanın öncesi ve sonrası olarak bölünür ve kalanlar yine permütasyon olmalıdır.
  -

- OCL'nin yeri
  - OCL, UML diyagramlarının anlatamadığı anlamsal ayrıntıları verir. FOL ile aynı şeyi söyler ama programlama diline benzeyen sözdizimi sayesinde daha kolay okunur ve CASE araçları tarafından desteklenir. Hoca, OCL'yi yapısal ve davranışsal diyagramların vazgeçilmez tamamlayıcısı olarak tanımlar. Durum diyagramlarındaki geçiş koşulları da OCL ile yazılacaktır.
  - Hoca, güçlü tiplemeyi açıklarken bir değişkene önce tam sayı, sonra metin atanabilen dilleri (JavaScript, Python) örnek verir.
- Kütüphane örneğinde OCL
  - Önceki derste kurulan model hatırlatılır: ziyaretçiler başlıkları talep eder (Don Quixote'un beş kopyası olabilir, istenen belirli bir kopya değil başlıktır); ödünç bir ilişki sınıfıdır (uzatıldı mı, teslim tarihi, türetilen ceza); parçalar ödünç verilebilir (Kitap, DVD) ve verilemez (başvuru kitabı, dergi) olarak ayrılır; Para, Tarih ve İşletim Sistemi `<<utility>>` sınıflarıdır.
  - `oduncAldiklari()` bir küme döndürür: aynı başlıktan üç ayrı kopya alınabilir ama raftaki belirli bir kopya aynı anda iki kez alınamaz. Sonuç, ziyaretçiden ödünç ilişki sınıfı üzerinden ulaşılan ödünç verilebilir parçaların tamamıdır; buna hem henüz getirilmeyenler hem de geç getirilenler dahildir.
  - `oduncAl` için ön koşullar birden fazladır ve ayrı ayrı yazılabilir ya da `and` ile birleştirilebilir. Son koşulda `exists` kullanılır, çünkü bütün ödünç nesneleri için değil, yeni yaratılan bir ödünç nesnesi için bir şey garanti edilir: parçası `i`'dir, teslim tarihi bugünün tarihine parçanın ödünç periyodu eklenerek bulunur ve ödünç koleksiyonu eski hâline bu nesnenin eklenmiş hâlidir. İkinci son koşul, ziyaretçi bu başlığı talep ettiyse talebi listeden `reject` ile düşürür; böylece sıradaki ziyaretçi listenin başına gelir.
  - Yaş, türetilen bir özelliktir: işletim sisteminden alınan bugünün tarihinden doğum tarihi çıkarılır. Böylece yaş her sorgulandığında güncel hesaplanır.
- **Ders sonrası notlar:**
  - Hoca 1, 2, 3, 4 karşı örneğini "her eleman X'te aynı sayıda yer almalıdır" koşulunu aşan bir çıktı olarak verdi. Slayttaki cümle Y'deki her eleman için yazıldığından, harfiyen okunursa 4 (X'te 0 kez, Y'de 1 kez) bu koşulu zaten sağlamaz; karşı örnek büyük olasılıkla koşulun yalnız X'teki elemanlar için okunduğu hâli için düşünülmüştür. Python ile denendi: 3, 2, 1 girdisi için yalnız 1, 2, 3 çıktısı permütasyon koşulunu sağlar.
  - Hoca güçlü tiplemeyi anlatırken Python'u zayıf tipli, C'yi güçlü tipli örnek verdi. Python dinamik tiplidir ama büyük olasılıkla güçlü tipli sayılır (`1 + "a"` hata verir); C ise statik tiplidir ama örtük dönüşümleri yüzünden çoğu kaynakta zayıf tipli kabul edilir. Hocanın kastettiği büyük olasılıkla statik ve dinamik tipleme ayrımıdır.
  - Karekök OCL örneğinde dönüş tipi tam sayı olarak anlatıldı. `self = result * result` son koşulu yalnız tam kareler için sağlanabilir; genel bir karekök için dönüş tipinin Real olması büyük olasılıkla daha uygundur.
  - `self.Order.Check.bankID->size()` hocanın anlatımında "kaç farklı banka çeki" olarak okundu. Gezinti sonucu tekrarlı bir koleksiyon olduğundan `size()` çek sayısını verir; farklı banka sayısı için büyük olasılıkla `->asSet()->size()` gerekir.
  - Yaş özelliği slaytta `Date` tipiyle yazıldı; önceki dersin modelinde `/yaş: Integer` idi. Tarih farkından yaş elde etmek için büyük olasılıkla Integer tipi ve yıl hesabı gerekir.
  - Gereksinim 4 slaytında "12 yaş altındaki" denirken OCL'de `yas <= 12` kullanıldı; bu, 12 yaşındakileri de kapsar.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.